
CS 301

High-Performance Computing

Assignment 8 - Parallel Interpolation with & Particle Mover using MPI + OpenMP

Diya Patel (202301216)

Yesha Joshi (202301462)

April, 2026

0 Contents

1	Introduction	2
2	Implementation Approach and Parallel Design	2
2.1	Overview of the Hybrid Strategy	2
2.2	Race-Condition Handling in Interpolation	2
2.3	Pseudocode	3
2.4	Illustrative Diagram	4
3	Input Configurations	5
4	Performance Results	5
4.1	Execution Time vs. Cores	5
4.2	Speedup vs. Cores	8
4.3	Maximum Speedup Across Configurations	10
4.4	Parallel Efficiency	11
4.5	Phase-Level Timing Breakdown	13
4.6	Bottleneck Analysis	16
4.7	Heatmaps of Total Time vs. $\text{MPI} \times \text{OMP}$	18
4.8	Serial vs. Best Parallel Time	21
5	Analysis and Questions	21
5.1	Q1: Pseudocode and Illustrative Diagram	21
5.2	Q2: Parallel Implementation Approach and Race-Condition Handling . . .	21
5.3	Q3: Speedup and Execution Time Graphs, Login vs. Compute Nodes . . .	22
5.4	Q4: Parallel Efficiency and Scalability	22
5.5	Q5: Comparison with Serial Version and Maximum Speedup	22
5.6	Q6: Explanation of Observed Speedup	23
5.7	Q7: Further Optimisations with Unlimited HPC Resources	23
5.8	Q8: Load Imbalance and Performance Bottlenecks	24
5.9	Q9: Alternative Approaches to Improve Performance	24
5.10	Q10: Alternative Data Structures and MPI-OMP Approaches	24
5.11	Q11: Interpolation vs. Mover Phase Bottleneck	25
6	Conclusion	25

1 Introduction

This report documents the design, implementation, and performance analysis of a hybrid **MPI + OpenMP** parallel code for scatter-to-mesh interpolation combined with a particle mover (reverse interpolation) step. The work extends the previous OpenMP-only assignment by adding an MPI communication layer that enables computation to span multiple nodes on the HPC cluster.

The problem involves a set of N scattered points (x_i, y_i, f_i) randomly distributed within a $[0, 1] \times [0, 1]$ domain whose function values ($f_i = 1$) are interpolated onto a structured $M_x \times M_y$ mesh using bilinear (linear) interpolation. After each iteration the mesh values are normalised, reverse-interpolated back to particles (the *mover*), and particle positions are updated. Particles that leave the domain are marked inactive.

The complete pipeline is:

1. Particle $\rightarrow$ grid interpolation (scatter)
2. MPI **Reduce** to assemble the global mesh on rank 0
3. MPI **Bcast** to distribute the global mesh to all ranks
4. Mesh normalisation to $[-1, 1]$
5. Grid $\rightarrow$ particle reverse interpolation (mover) + position update
6. Mesh de-normalisation

This pipeline repeats for `Maxiter` = 10 iterations for each of five input configurations.

2 Implementation Approach and Parallel Design

2.1 Overview of the Hybrid Strategy

The parallelisation combines two levels:

- **MPI (inter-node)**: Particles are distributed across MPI ranks using `MPI_Scatterv`. Each rank owns a contiguous chunk of the particle array and maintains a private copy of the mesh (`local_mesh`).
- **OpenMP (intra-node)**: Within each rank, multiple threads operate on the local particle subset using thread-private mesh buffers (`thread_mesh[tid]`) to eliminate write conflicts, then reduce into `local_mesh` serially.

2.2 Race-Condition Handling in Interpolation

The central challenge is that multiple particles may update the same grid cell simultaneously. Two standard strategies exist: atomic operations and thread-private buffers. We use **thread-private buffers (privatisation)** because:

- It completely eliminates write conflicts without serialising hot grid cells.
- Cache lines are not shared between threads, avoiding false sharing.
- The post-loop reduction over `thread_mesh` arrays has predictable memory access and can be optimised by the compiler.

Each OpenMP thread t writes to its own array `thread_mesh[t][0..grid_size-1]`. After the parallel region, a serial loop sums them into `local_mesh`. An MPI Reduce (SUM) then assembles the final global mesh on rank 0, followed by a Bcast to share it with all ranks before the mover phase.

2.3 Pseudocode

```

1  MPI_Init()
2  rank, size ← MPI_Comm_rank/size()
3
4  IF rank == 0: points ← load_input(file)
5  MPI_Bcast(GRID_X, GRID_Y, NUM_Points, Maxiter)
6
7  Scatter particles across ranks using MPI_Scatterv
8    local_points[0..local_n-1] ← chunk for this rank
9
10 Allocate local_mesh[grid_size], global_mesh[grid_size]
11 Allocate thread_mesh[num_threads][grid_size] // private buffers
12
13 MPI_Barrier(); total_start ← MPI_Wtime()
14
15 FOR iter = 0 to Maxiter-1:
16
17     //          Phase 1: Scatter interpolation (particle → mesh)
18
19     memset(local_mesh, 0)
20     memset(thread_mesh[all], 0)
21     #pragma omp parallel
22     {
23         tid ← omp_get_thread_num()
24         pmesh ← thread_mesh[tid]
25         #pragma omp for
26         FOR p = 0 to local_n-1:
27             IF local_points[p].is_void: CONTINUE
28             (i, j, lx, ly) ← cell_index(local_points[p])
29             Compute bilinear weights (w00, w10, w01, w11)
30             pmesh[j*stride + i] += w00
31             pmesh[j*stride + i+1] += w10
32             pmesh[(j+1)*stride + i] += w01
33             pmesh[(j+1)*stride+i+1] += w11
34         }
35     }
36     FOR t = 0 to num_threads-1:
37         local_mesh += thread_mesh[t] // serial reduction
38
39     //          Phase 2: MPI communication
40
41     MPI_Reduce(local_mesh → global_mesh, SUM, root=0)
42     MPI_Bcast(global_mesh, root=0)
43
44     raw_mesh ← copy(global_mesh)
45
46     //          Phase 3: Normalise mesh to [-1, 1]
47
48     (min_val, max_val) ← parallel_min_max(global_mesh)
49     #pragma omp parallel for

```

```

46  global_mesh[i] ← 2*(global_mesh[i]-min_val)/(max_val-min_val) - 1
47
48  //          Phase 4: Mover (mesh → particle)
49
50  #pragma omp parallel for
51  FOR p = 0 to local_n-1:
52    IF is_void: CONTINUE
53    (i, j, lx, ly) ← cell_index(local_points[p])
54    Fi ← bilinear_gather(global_mesh, i, j, lx, ly)
55    local_points[p].x += Fi * dx
56    local_points[p].y += Fi * dy
57    IF out_of_domain: local_points[p].is_void ← 1
58
59  //          Phase 5: Denormalise
60
61  #pragma omp parallel for
62  global_mesh[i] ← ((global_mesh[i]+1)*0.5)*(max_val-min_val)+min_val
63
64  total_end ← MPI_Wtime()
65  IF rank == 0: write_output(raw_mesh); print timings
66  MPI_Finalize()

```

Listing 1: Hybrid MPI+OpenMP Interpolation and Mover Pipeline

2.4 Illustrative Diagram

Figure 1 presents the data flow and inter-process communication pattern across MPI ranks in the hybrid MPI+OpenMP implementation.

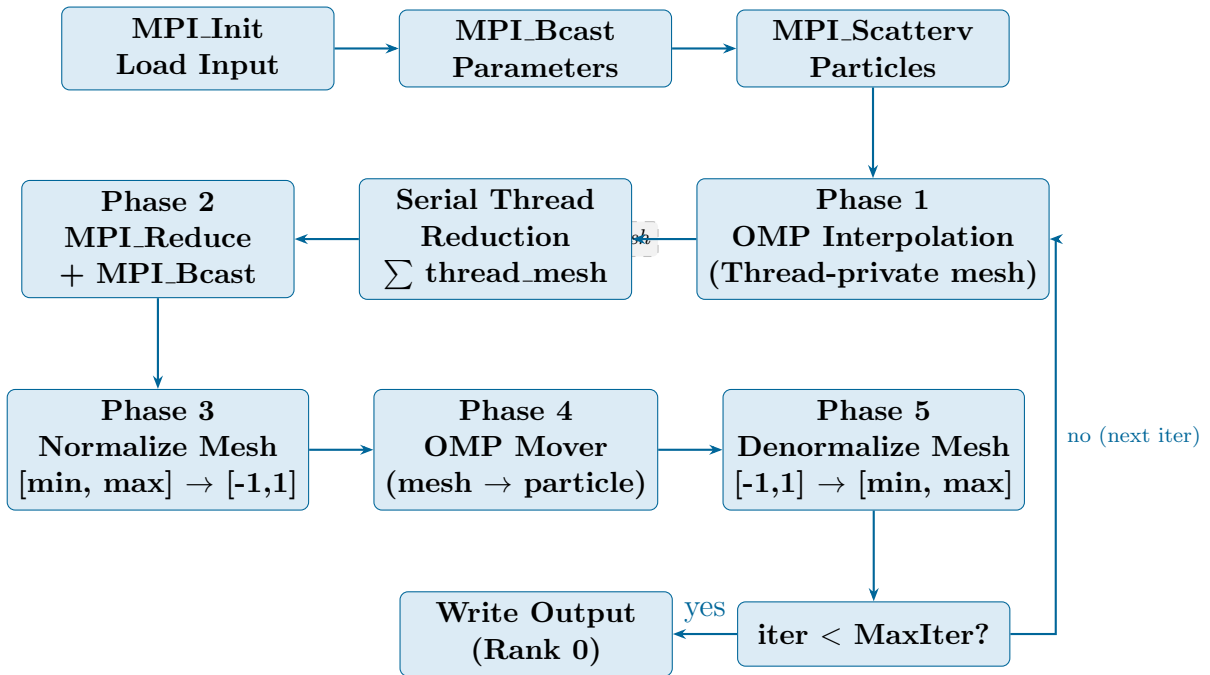


Figure 1: Hybrid MPI+OpenMP pipeline showing all five computational phases.

3 Input Configurations

Table 1 summarises the five problem configurations used for evaluation. All configurations use `Maxiter=10`.

Table 1: Problem configurations used for performance evaluation.

Config	Label	Grid ($N_x \times N_y$)	Particles	Grid cells
1 (a)	CONFIG1	250×100	0.9 M	25,351
2 (b)	CONFIG2	250×100	5 M	25,351
3 (c)	CONFIG3	500×200	3.6 M	100,701
4 (d)	CONFIG4	500×200	20 M	100,701
5 (e)	CONFIG5	1000×400	14 M	401,401

4 Performance Results

Experiments were performed on the HPC compute nodes **gics1–gics4**. The number of total cores (MPI ranks $\times$ OMP threads) was varied from 2 to 64, doubling each time. For each configuration the baseline serial time is taken as the 1-MPI-rank, smallest-OMP run.

4.1 Execution Time vs. Cores

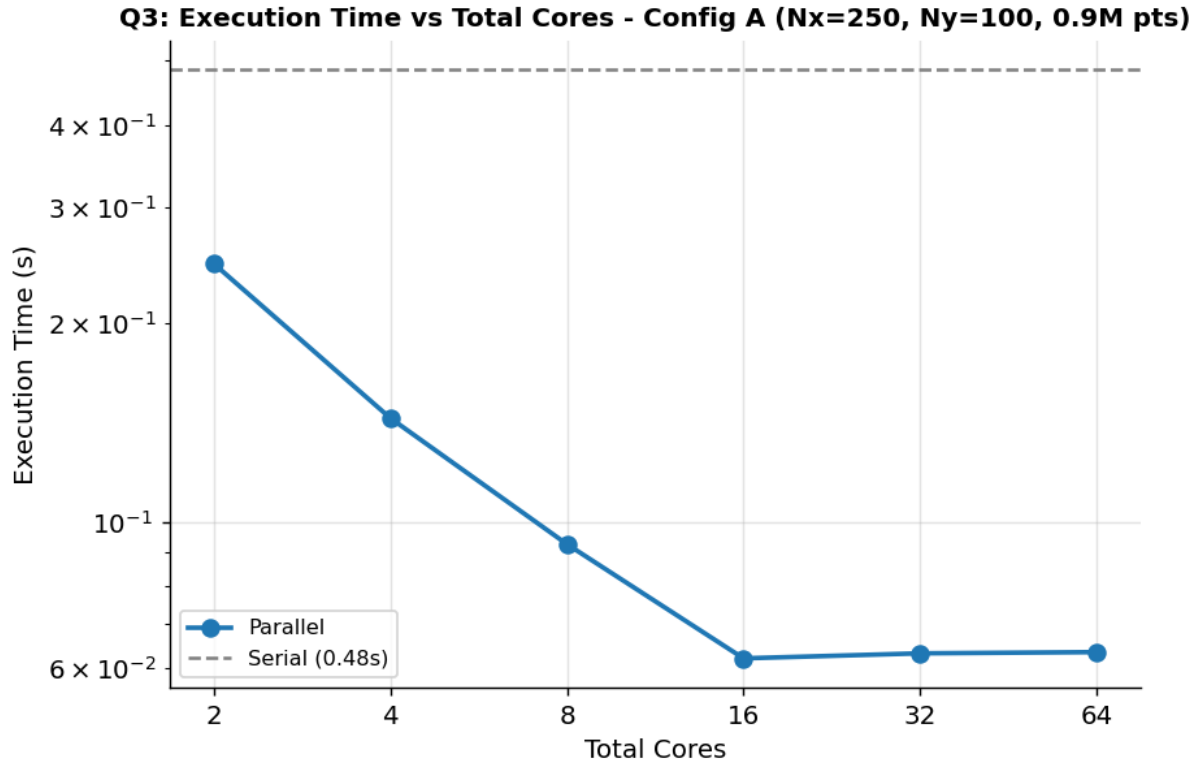


Figure 2: Execution time vs. number of cores — CONFIG1 (Nx=250, Ny=100, 0.9M particles).

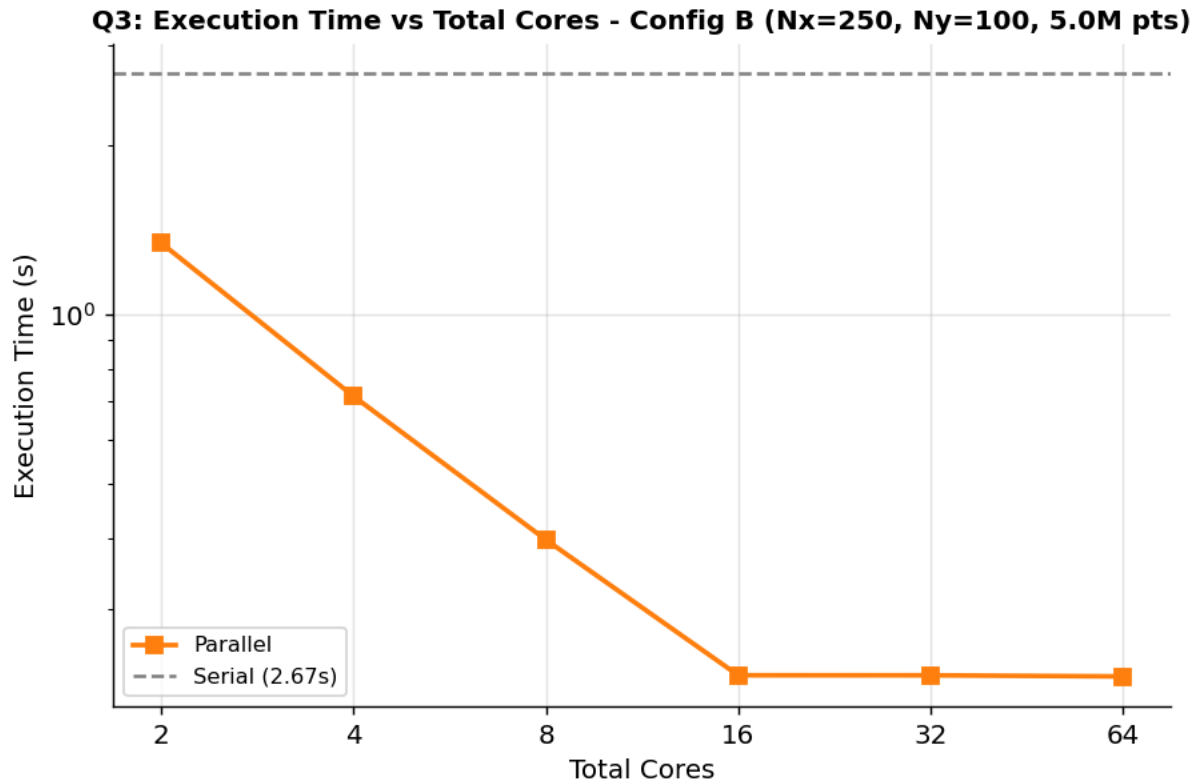


Figure 3: Execution time vs. number of cores — CONFIG2 (Nx=250, Ny=100, 5M particles).

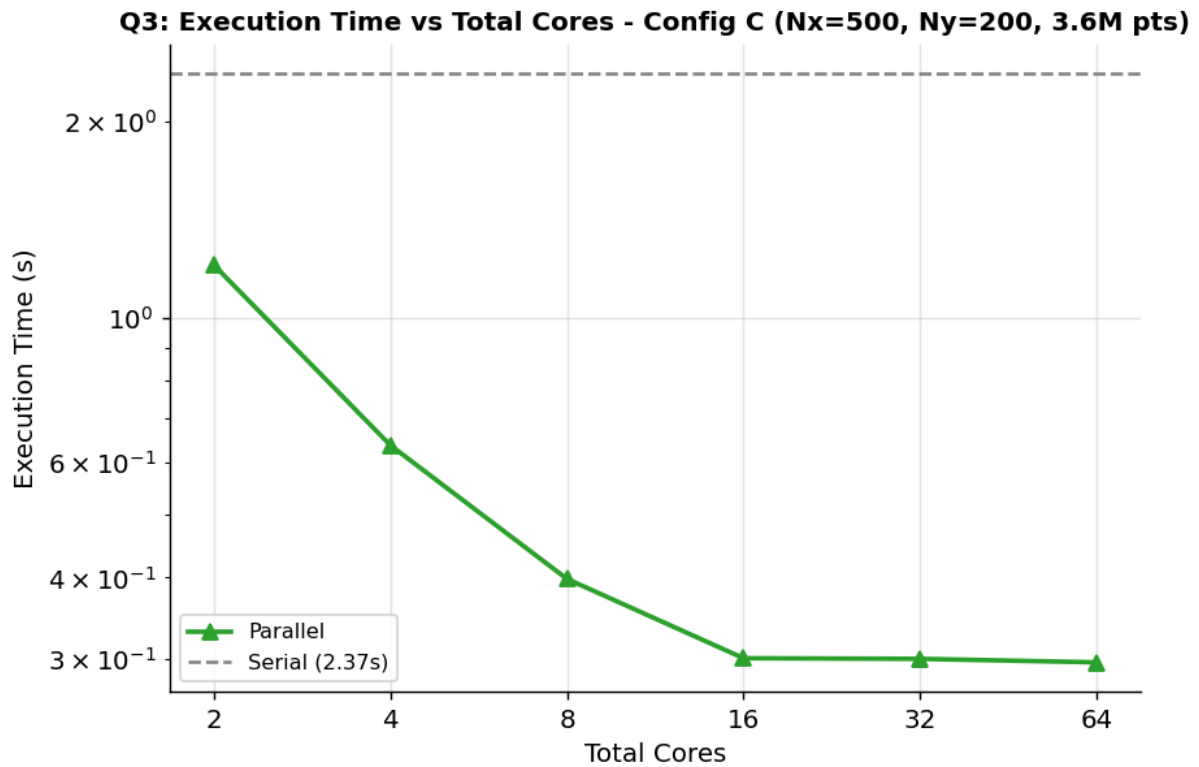


Figure 4: Execution time vs. number of cores — CONFIG3 (Nx=500, Ny=200, 3.6M particles).

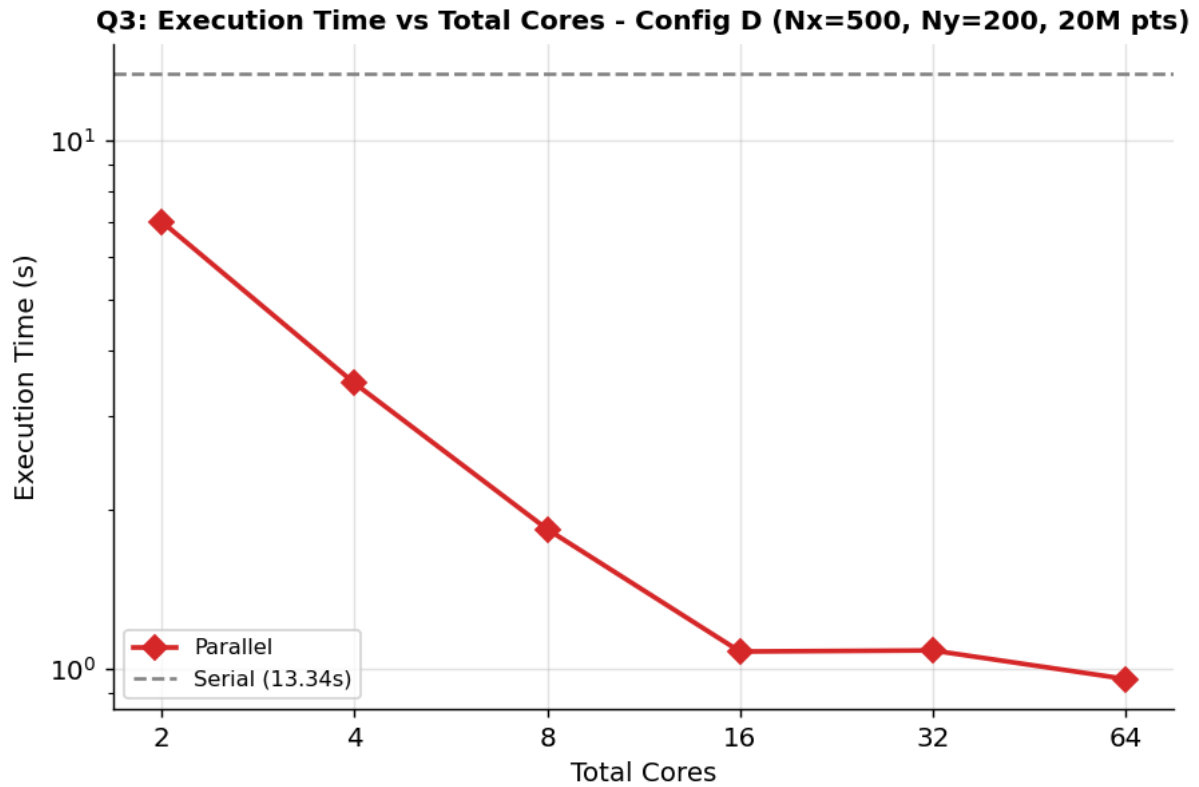


Figure 5: Execution time vs. number of cores — CONFIG4 (Nx=500, Ny=200, 20M particles).

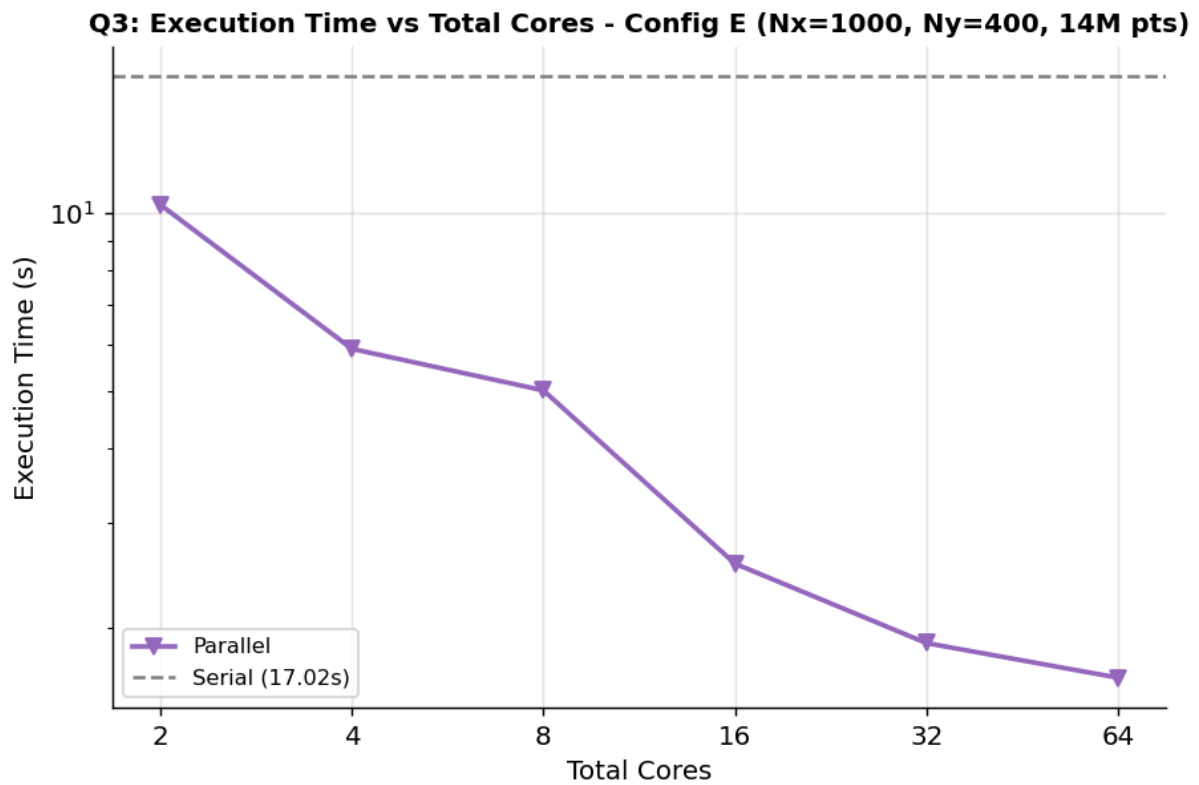


Figure 6: Execution time vs. number of cores — CONFIG5 (Nx=1000, Ny=400, 14M particles).

4.2 Speedup vs. Cores

Q3: Speedup vs Total Cores - Config A (Nx=250, Ny=100, 0.9M pts) - Max speedup: 7.81x

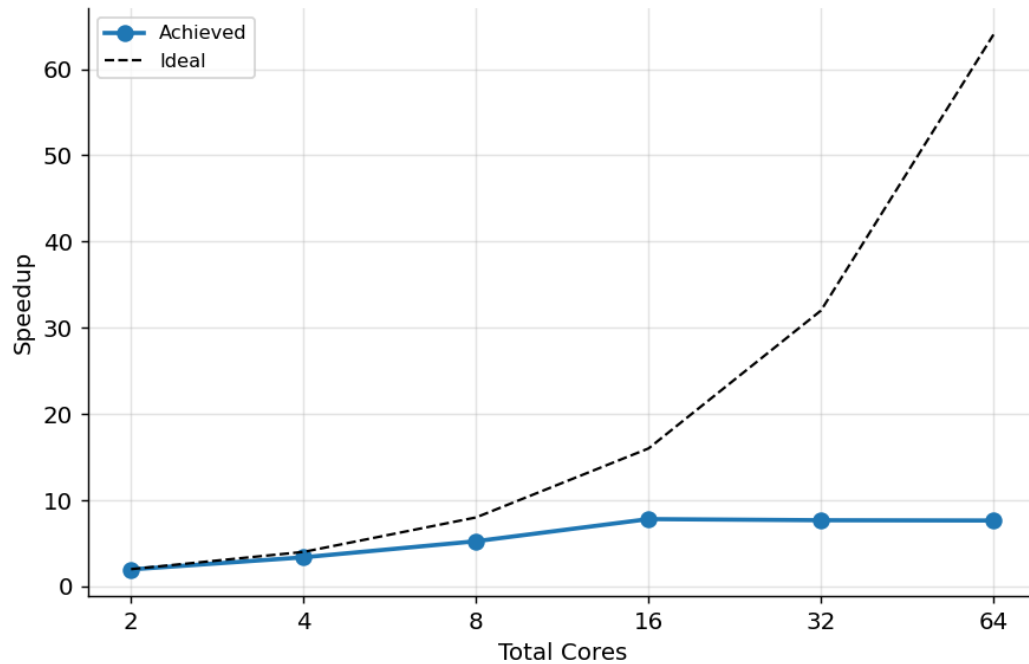


Figure 7: Speedup vs. number of cores — CONFIG1.

Q3: Speedup vs Total Cores - Config B (Nx=250, Ny=100, 5.0M pts) - Max speedup: 11.72x

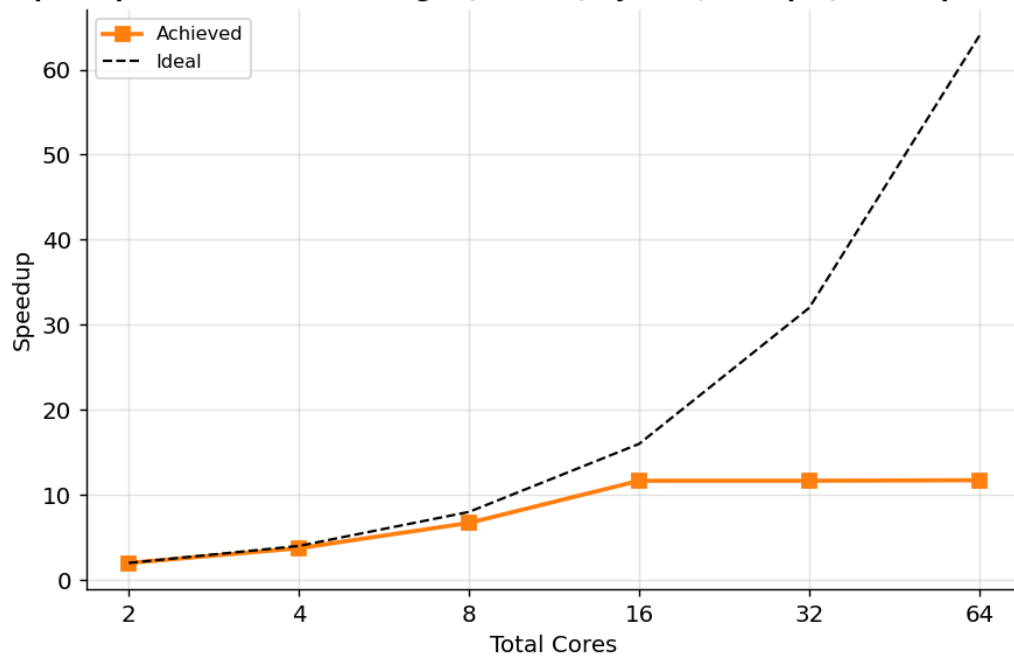


Figure 8: Speedup vs. number of cores — CONFIG2.

Q3: Speedup vs Total Cores - Config C (Nx=500, Ny=200, 3.6M pts) - Max speedup: 7.98x

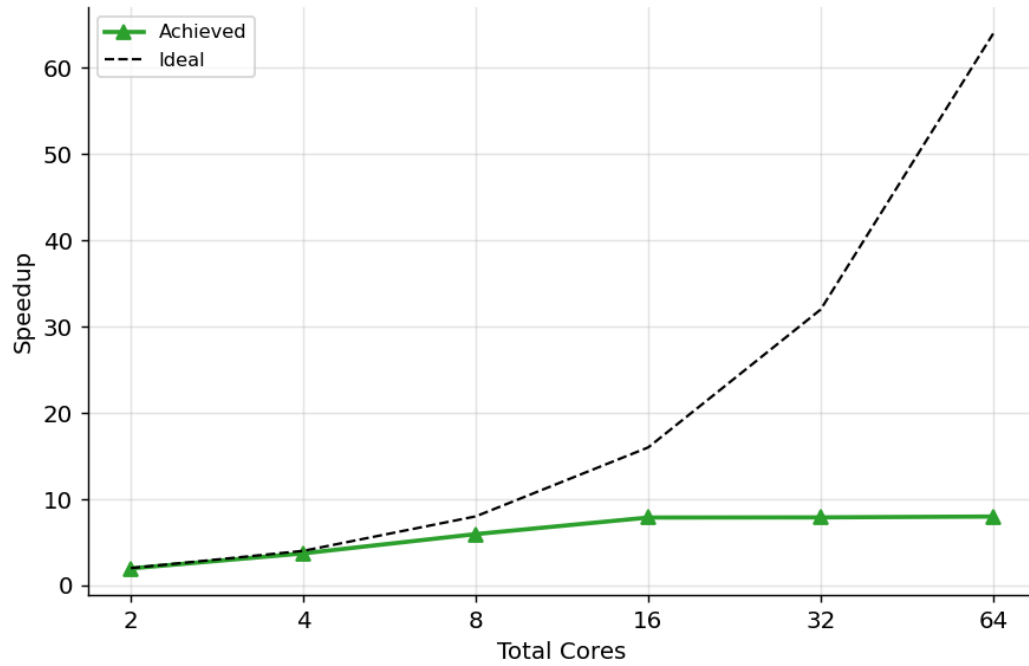


Figure 9: Speedup vs. number of cores — CONFIG3.

Q3: Speedup vs Total Cores - Config D (Nx=500, Ny=200, 20M pts) - Max speedup: 13.95x

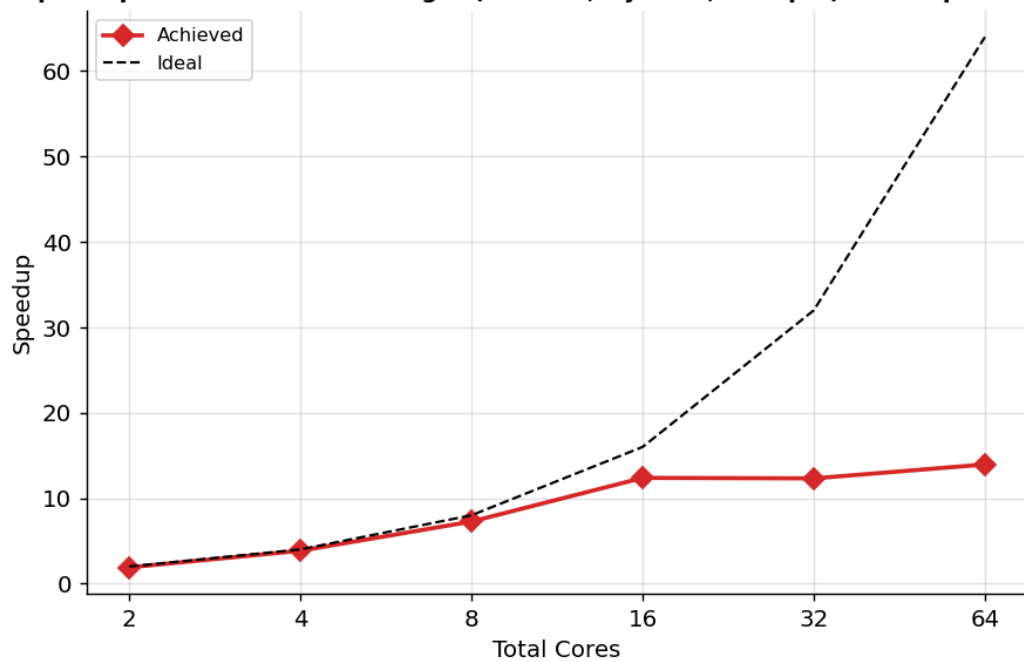
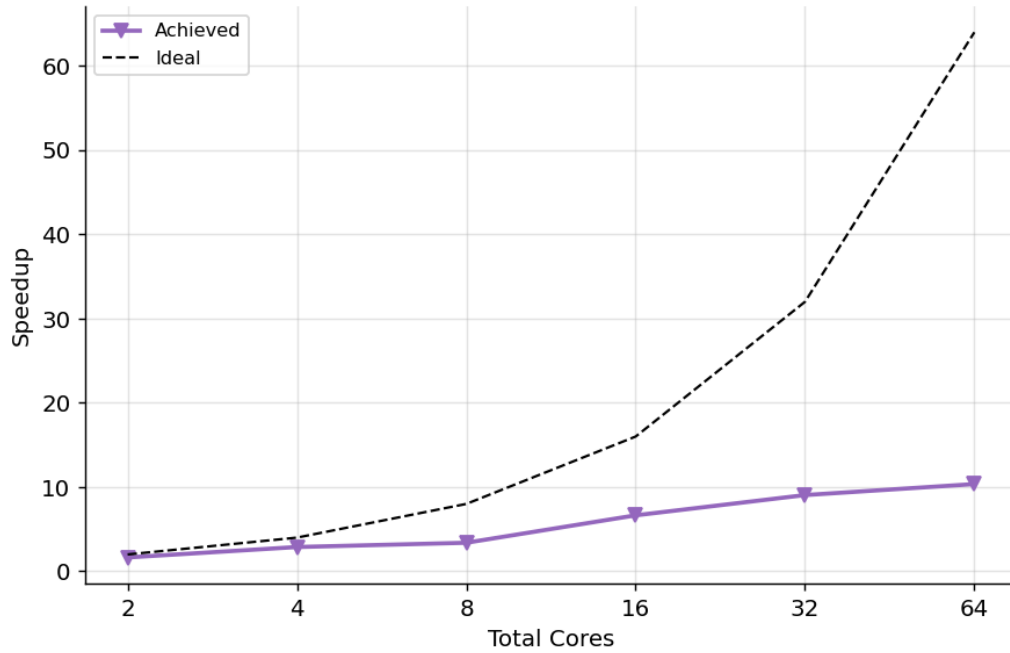
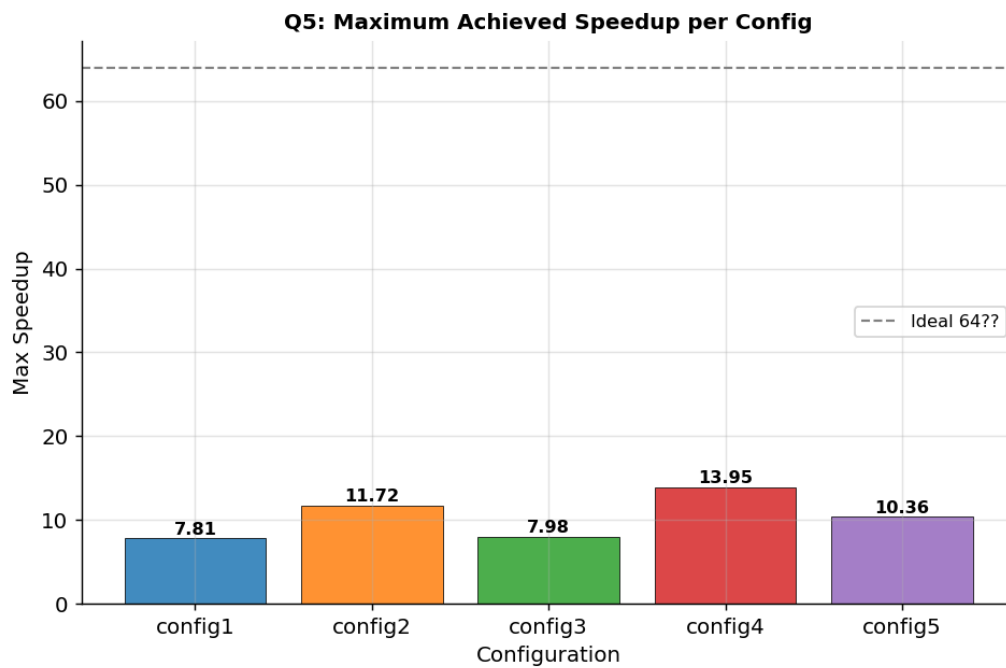


Figure 10: Speedup vs. number of cores — CONFIG4.

Q3: Speedup vs Total Cores - Config E (Nx=1000, Ny=400, 14M pts) - Max speedup: 10.36x**Figure 11:** Speedup vs. number of cores — CONFIG5.

4.3 Maximum Speedup Across Configurations

**Figure 12:** Maximum speedup achieved for each configuration across all core counts.

4.4 Parallel Efficiency

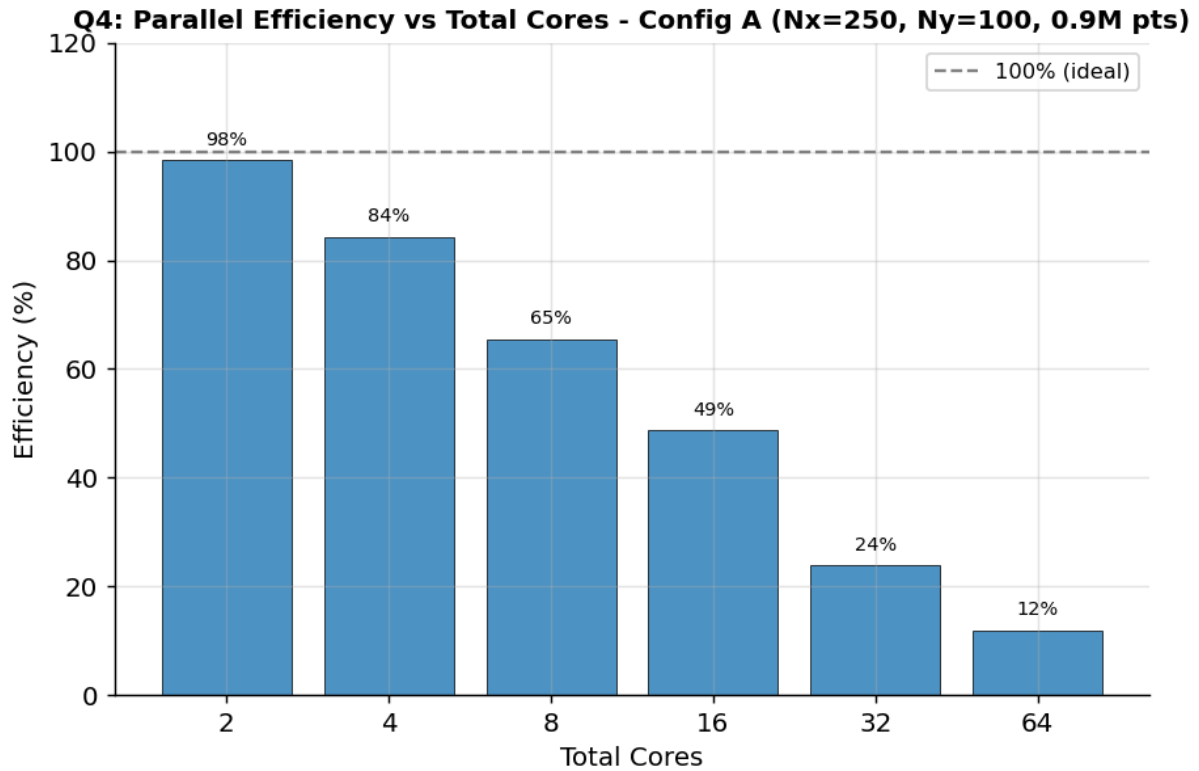


Figure 13: Parallel efficiency vs. number of cores — CONFIG1.

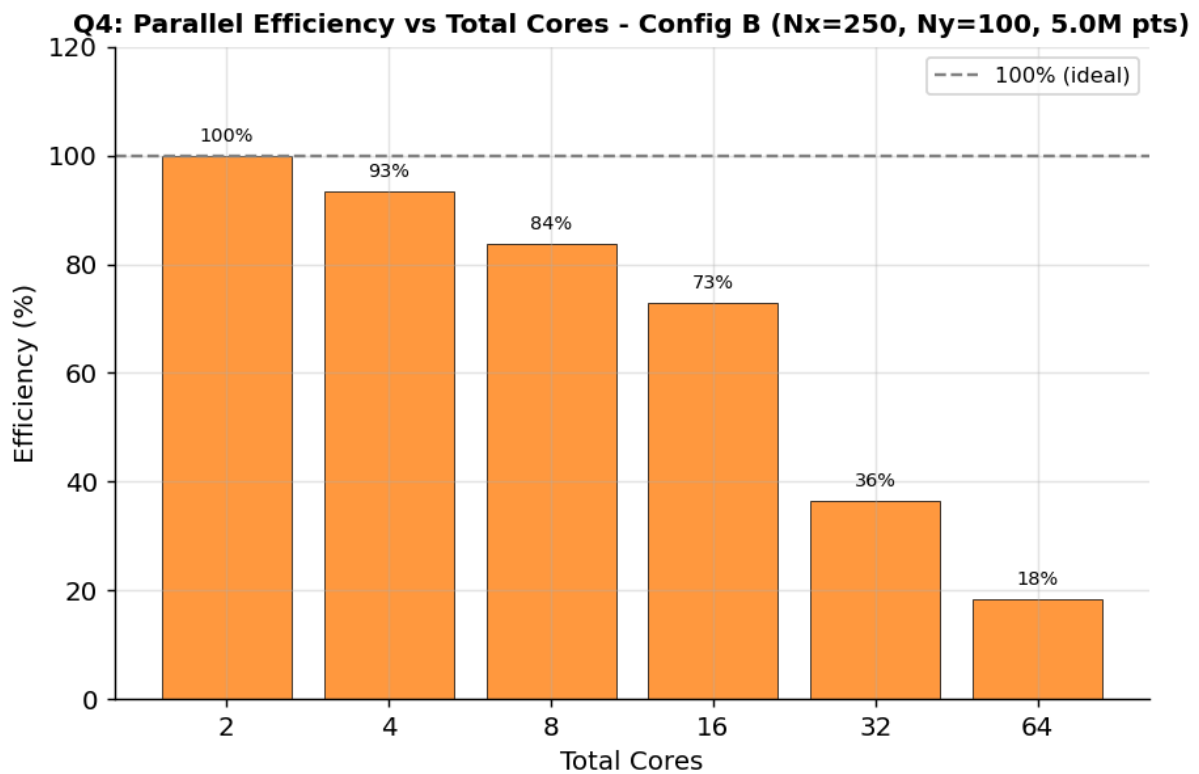


Figure 14: Parallel efficiency vs. number of cores — CONFIG2.

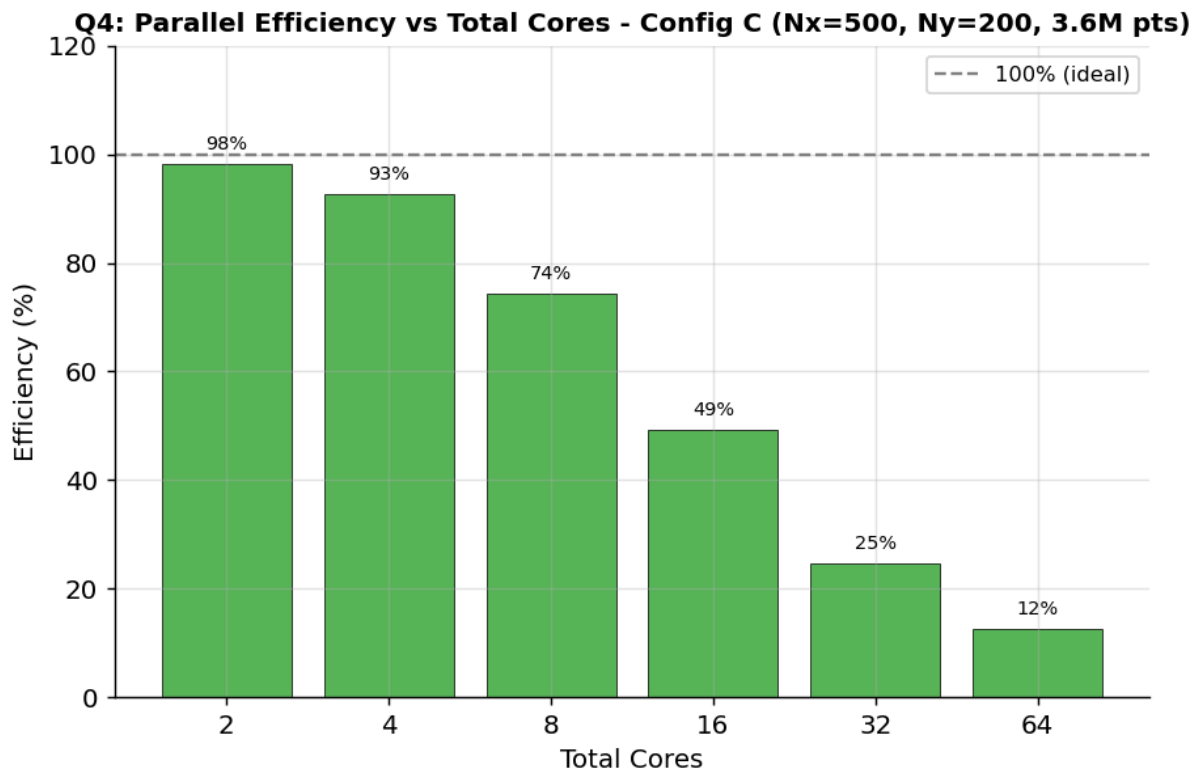


Figure 15: Parallel efficiency vs. number of cores — CONFIG3.

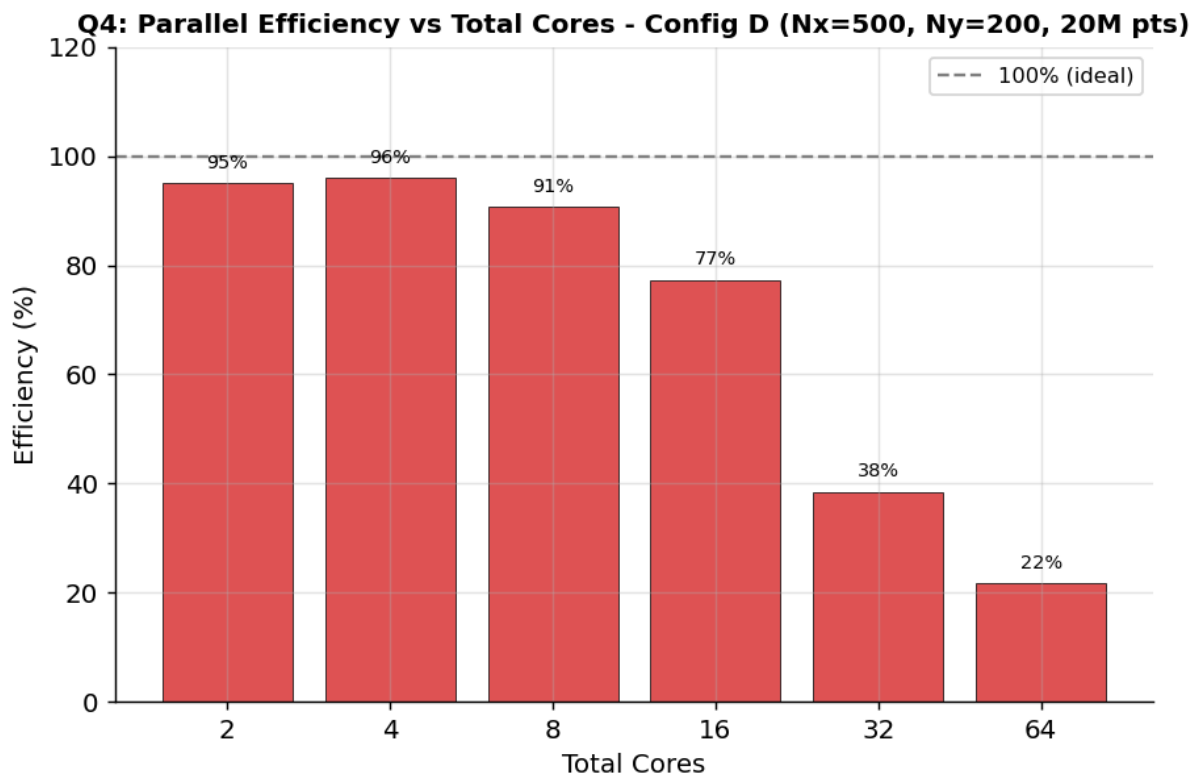


Figure 16: Parallel efficiency vs. number of cores — CONFIG4.

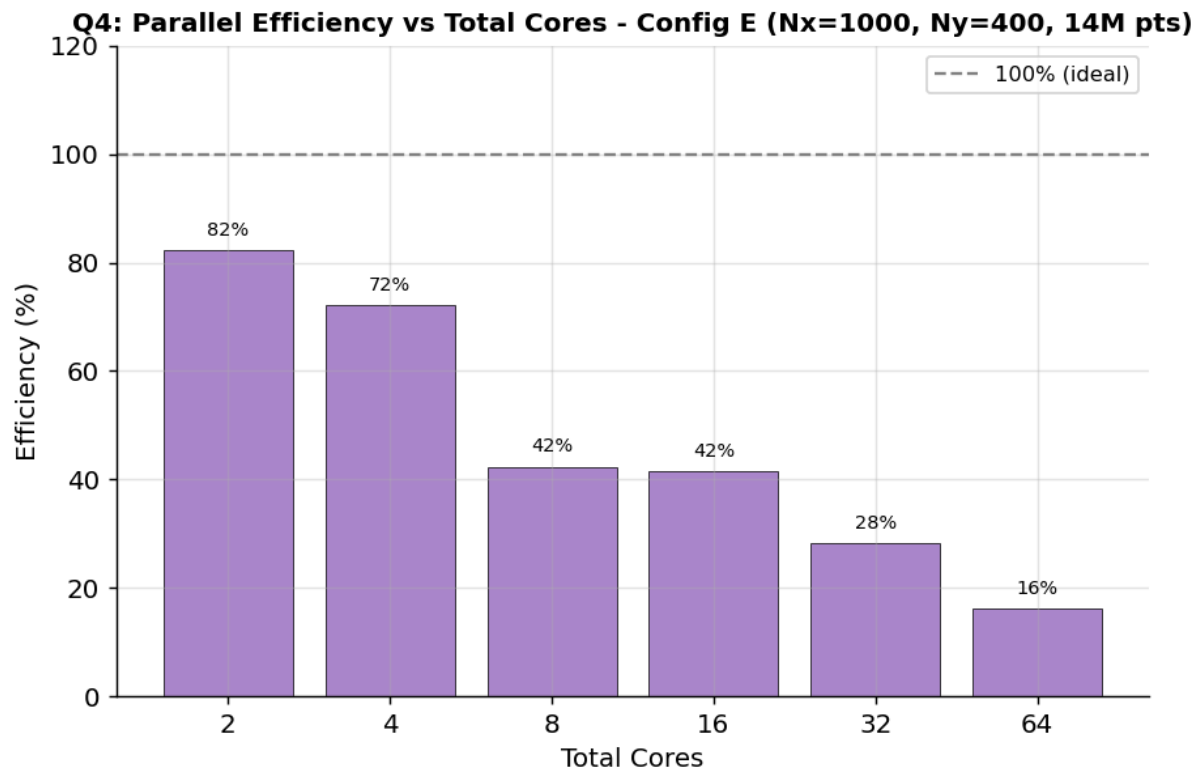


Figure 17: Parallel efficiency vs. number of cores — CONFIG5.

4.5 Phase-Level Timing Breakdown

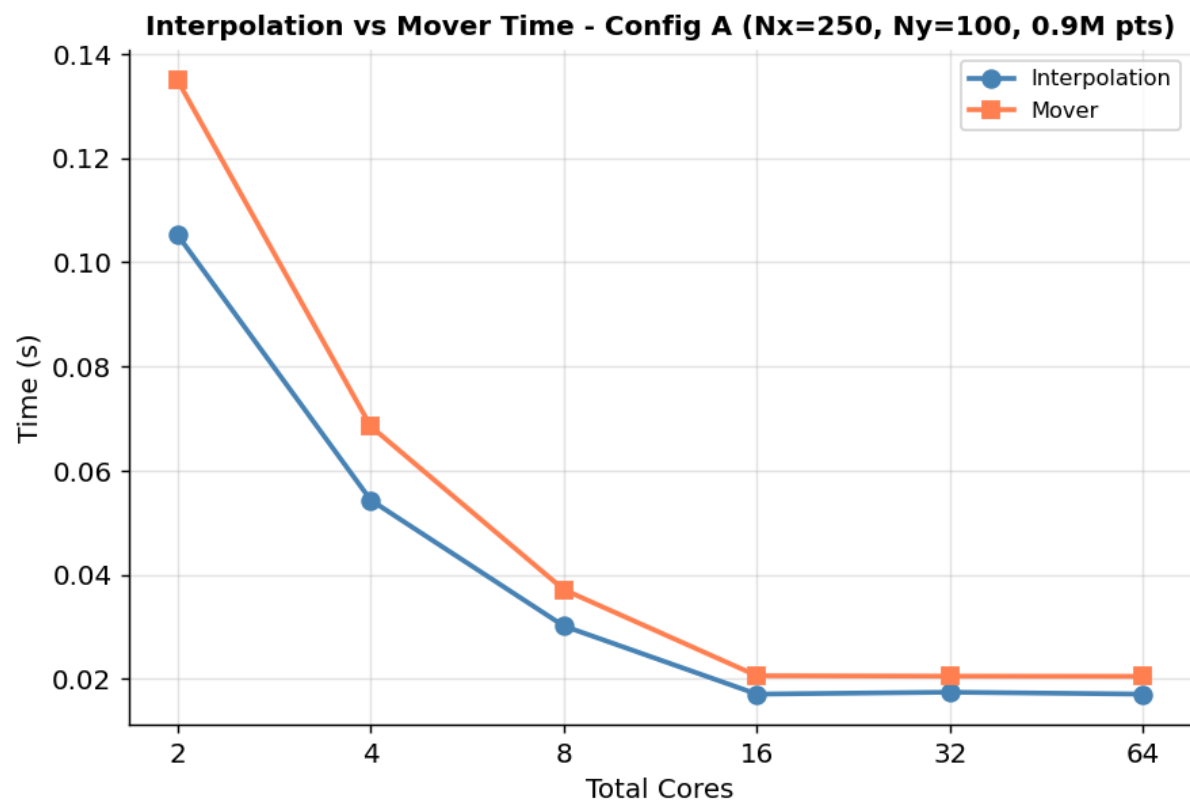


Figure 18: Phase-level timing breakdown — CONFIG1.

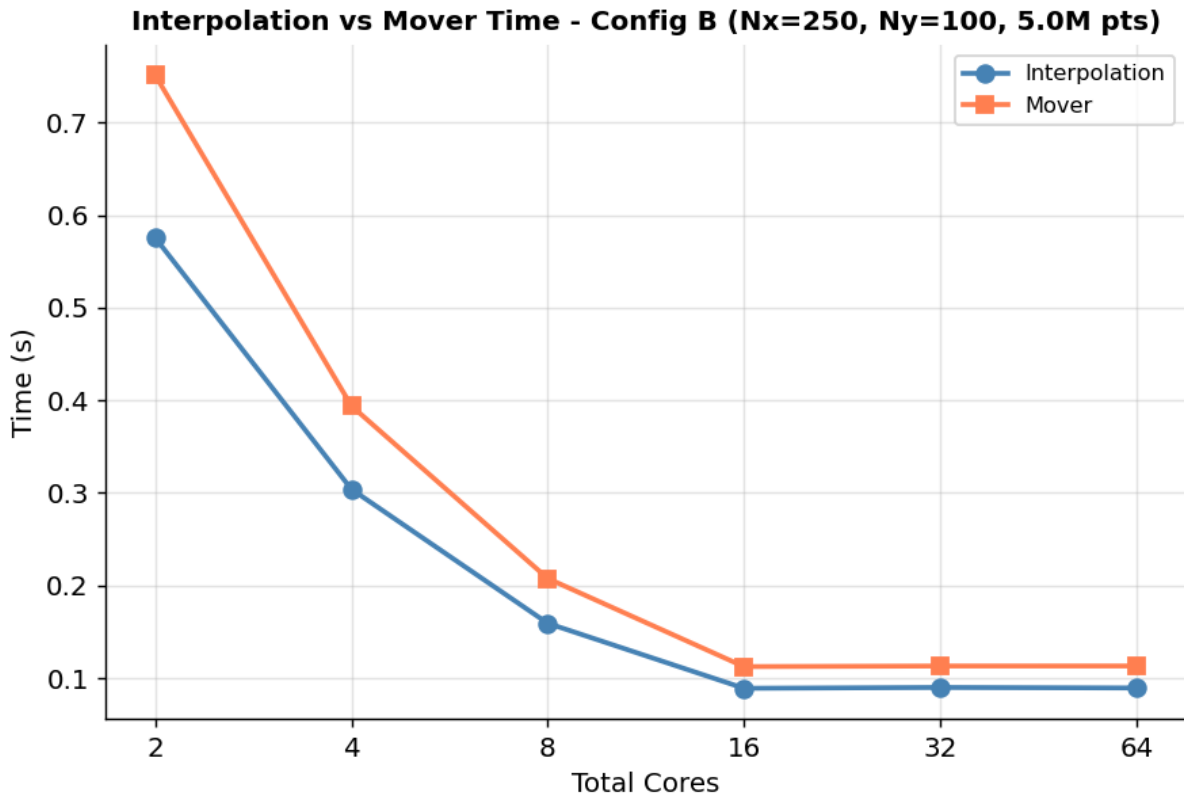


Figure 19: Phase-level timing breakdown — CONFIG2.

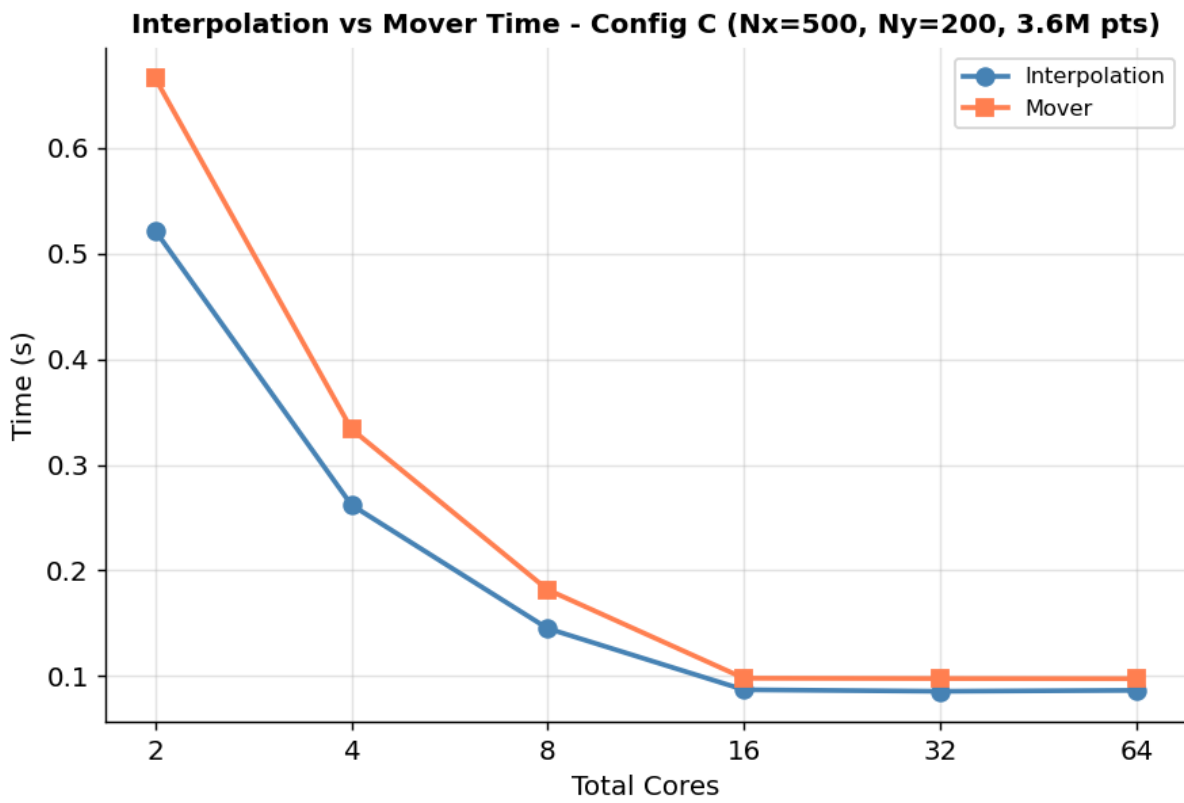


Figure 20: Phase-level timing breakdown — CONFIG3.

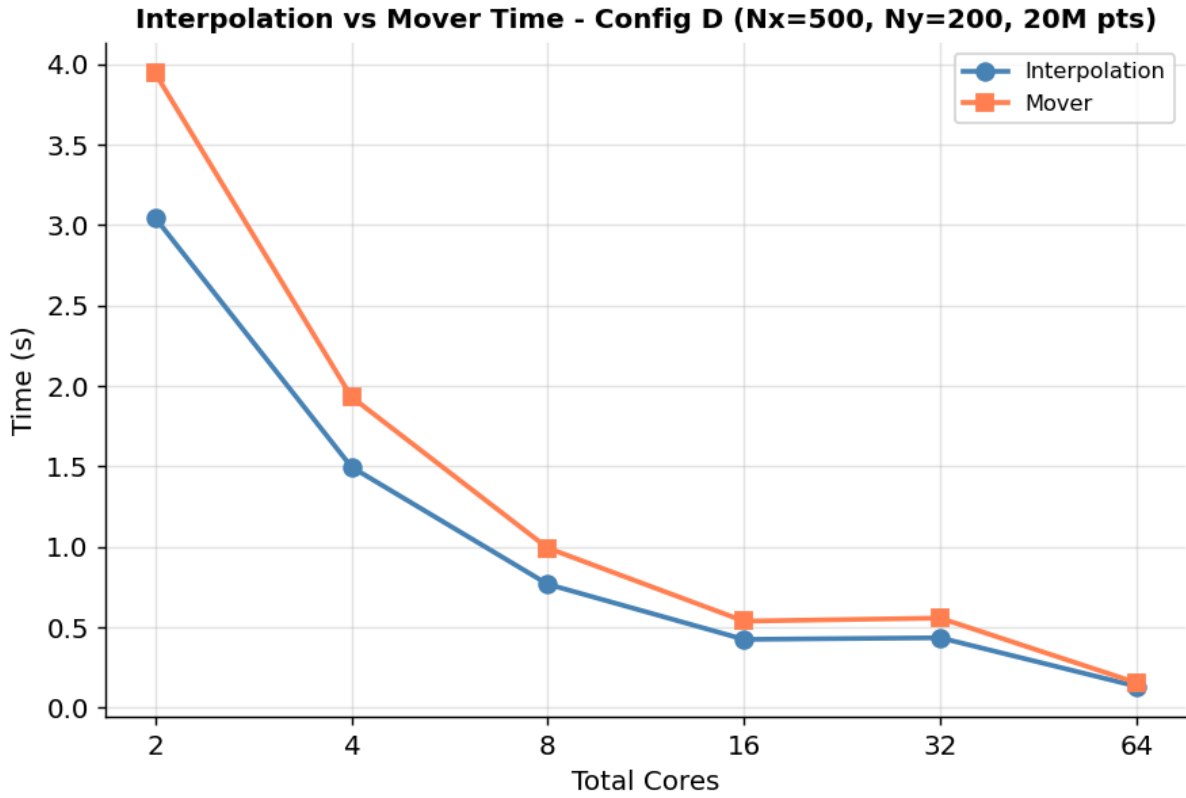


Figure 21: Phase-level timing breakdown — CONFIG4.

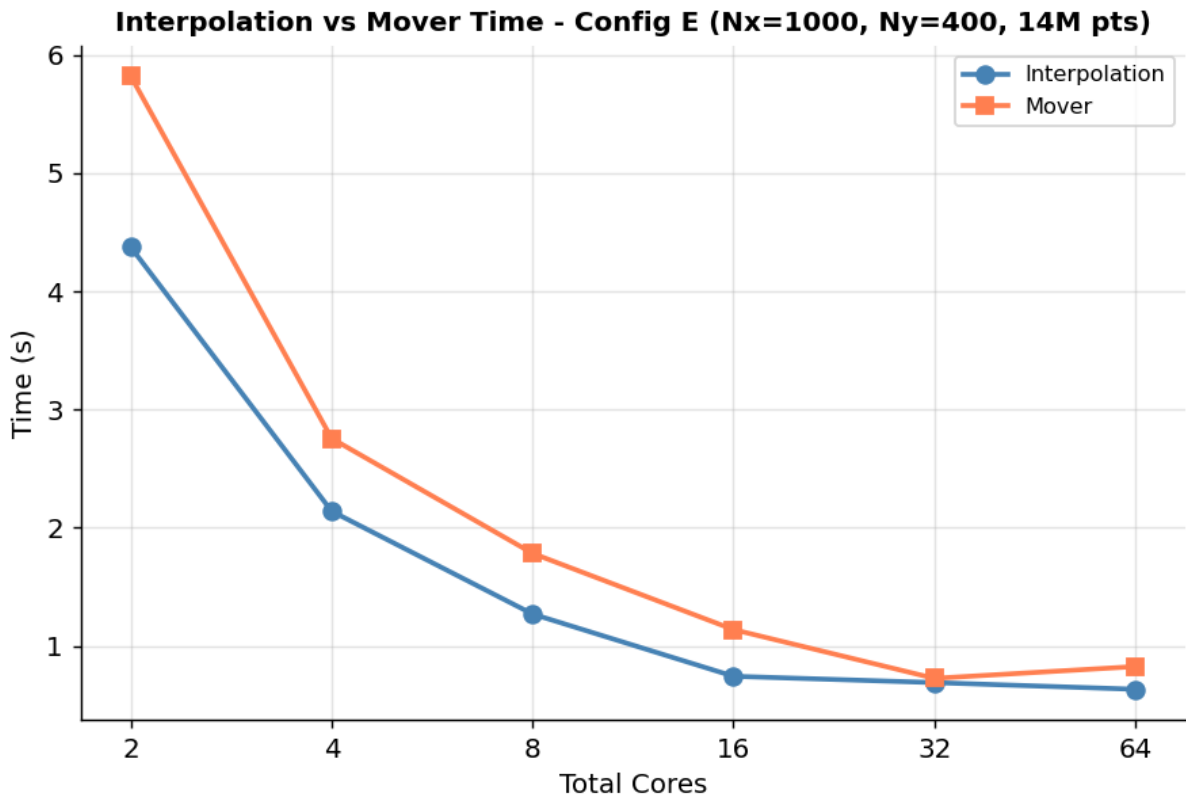


Figure 22: Phase-level timing breakdown — CONFIG5.

4.6 Bottleneck Analysis

Q8: MPI Communication Overhead vs Total Cores - Config A (Nx=250, Ny=100, 0.9M pts)

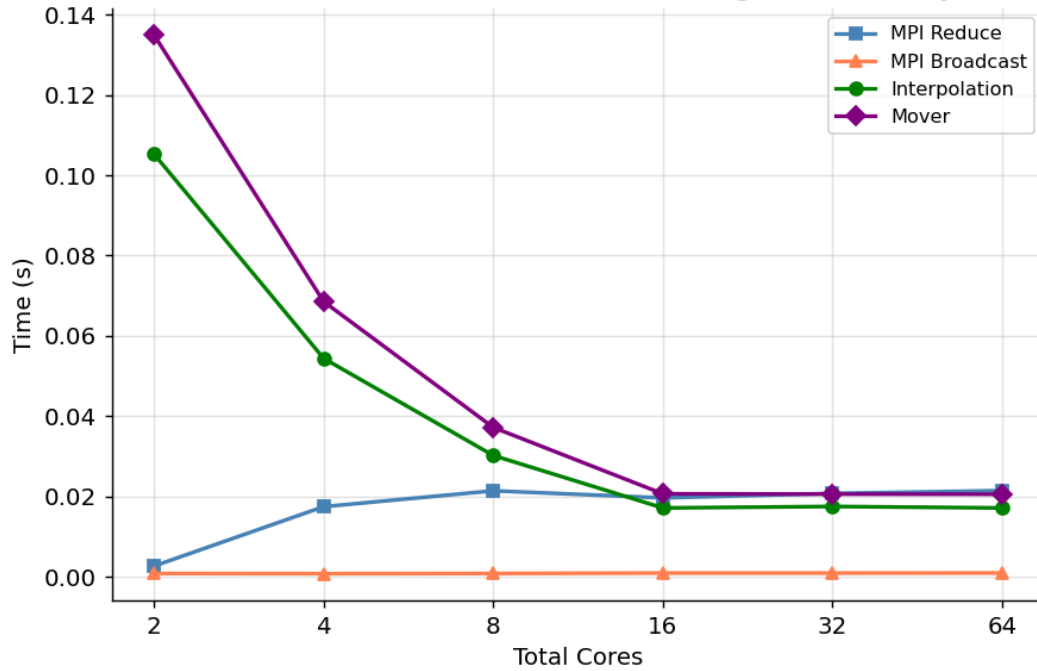


Figure 23: Bottleneck analysis showing dominant phases — CONFIG1.

Q8: MPI Communication Overhead vs Total Cores - Config B (Nx=250, Ny=100, 5.0M pts)

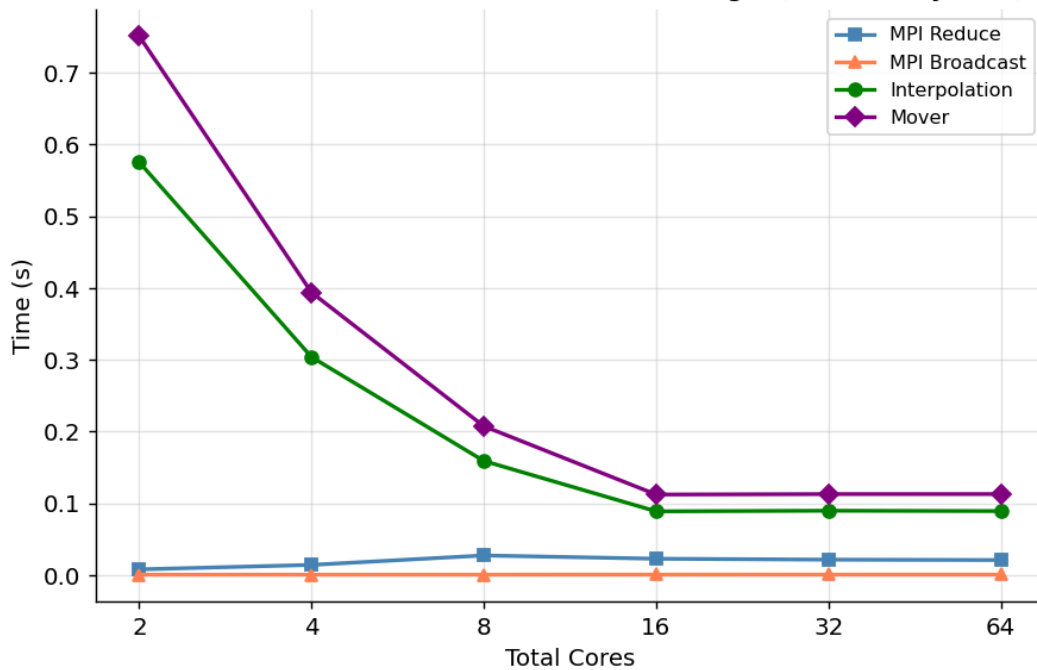


Figure 24: Bottleneck analysis — CONFIG2.

Q8: MPI Communication Overhead vs Total Cores - Config C (Nx=500, Ny=200, 3.6M pts)

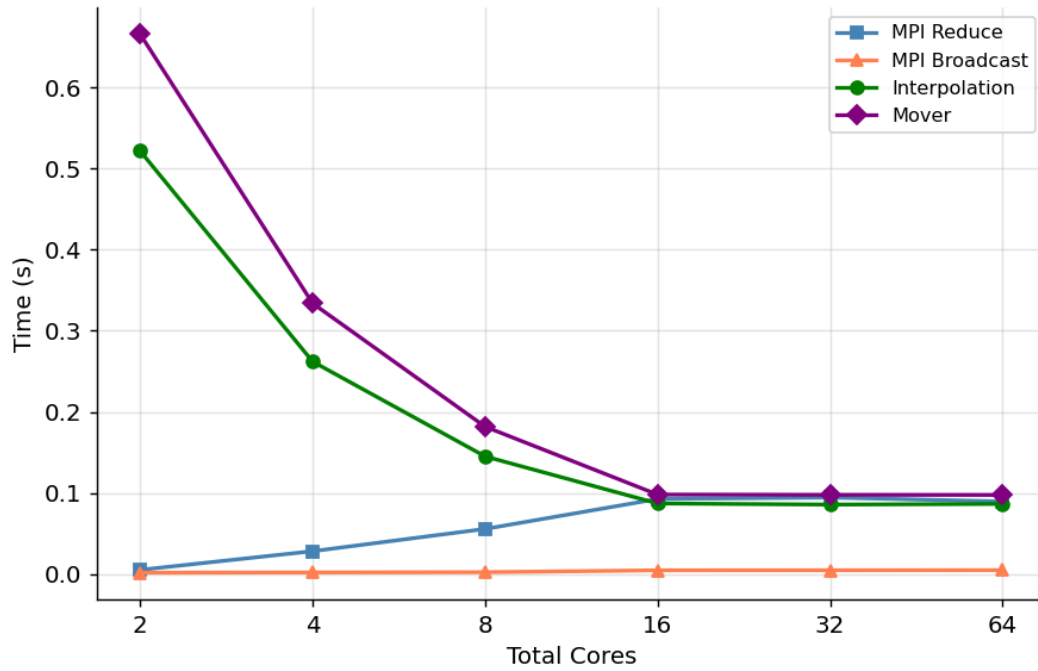


Figure 25: Bottleneck analysis — CONFIG3.

Q8: MPI Communication Overhead vs Total Cores - Config D (Nx=500, Ny=200, 20M pts)

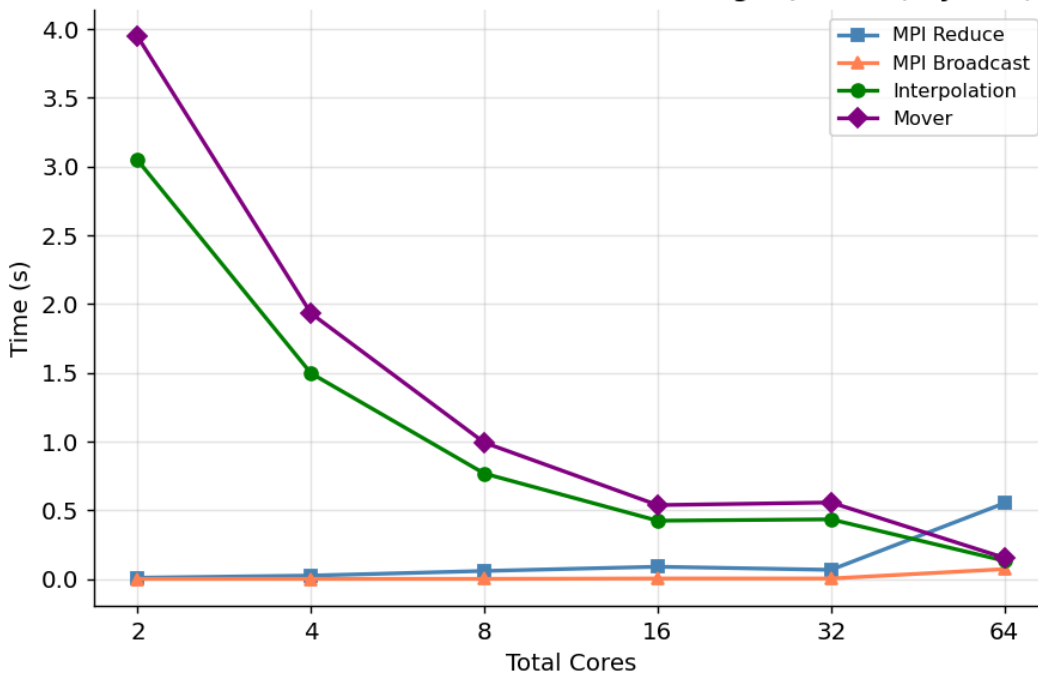


Figure 26: Bottleneck analysis — CONFIG4.

Q8: MPI Communication Overhead vs Total Cores - Config E (Nx=1000, Ny=400, 14M pts)

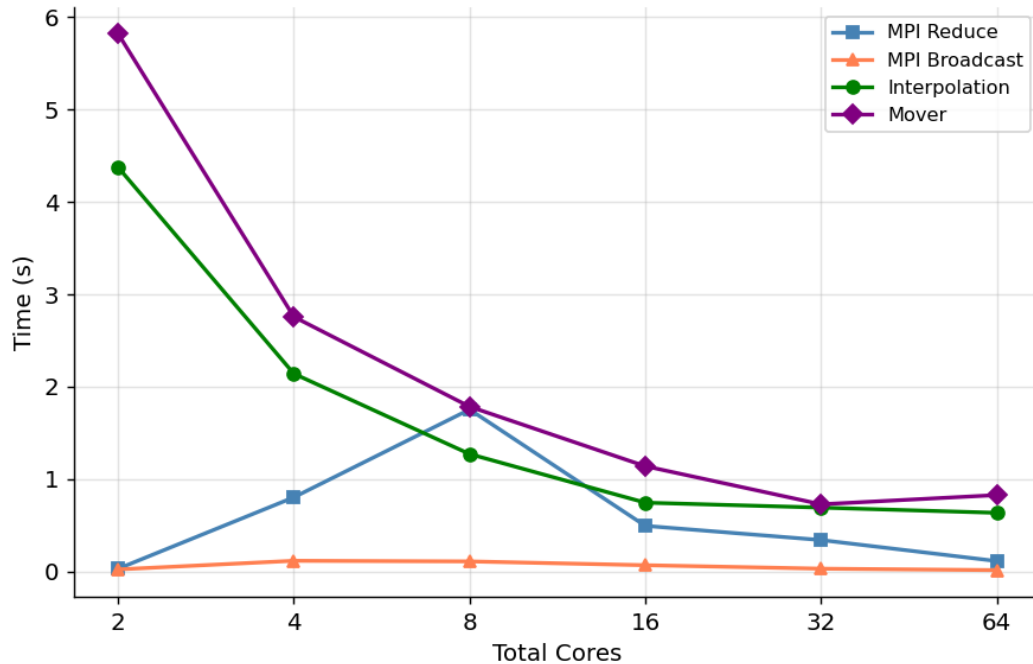


Figure 27: Bottleneck analysis — CONFIG5.

4.7 Heatmaps of Total Time vs. MPI × OMP

Total Execution Time Heatmap - Config A (Nx=250, Ny=100, 0.9M pts) - Lower is Better

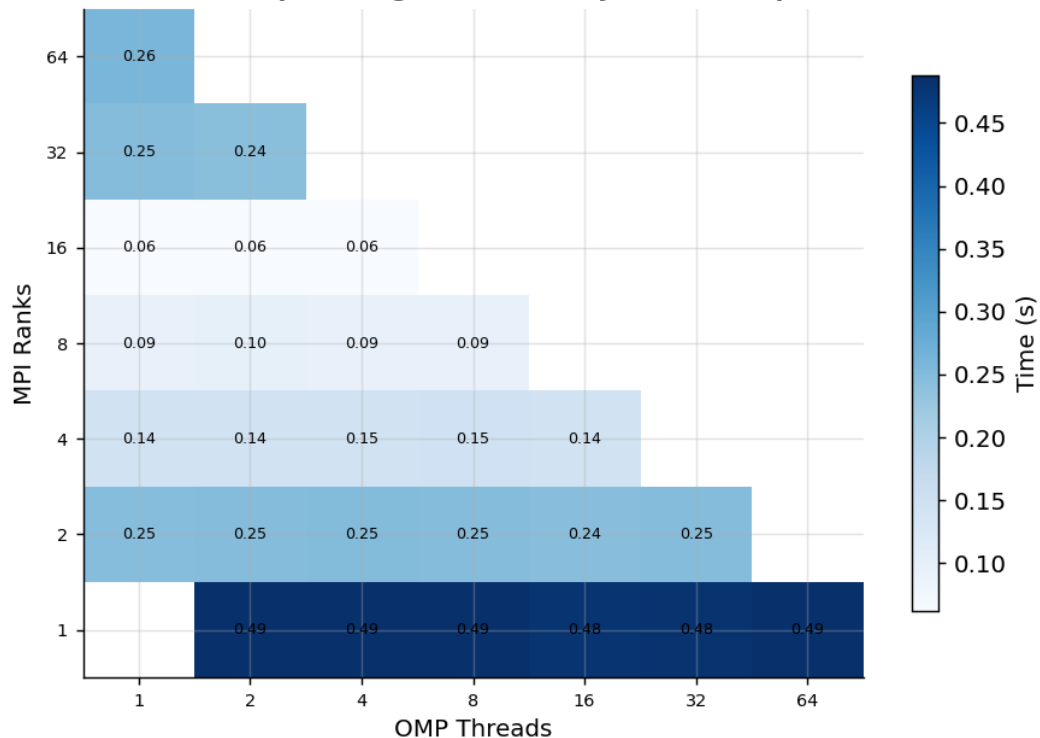


Figure 28: Heatmap of total execution time for different MPI×OMP combinations — CONFIG1.

Total Execution Time Heatmap - Config B (Nx=250, Ny=100, 5.0M pts) - Lower is Better

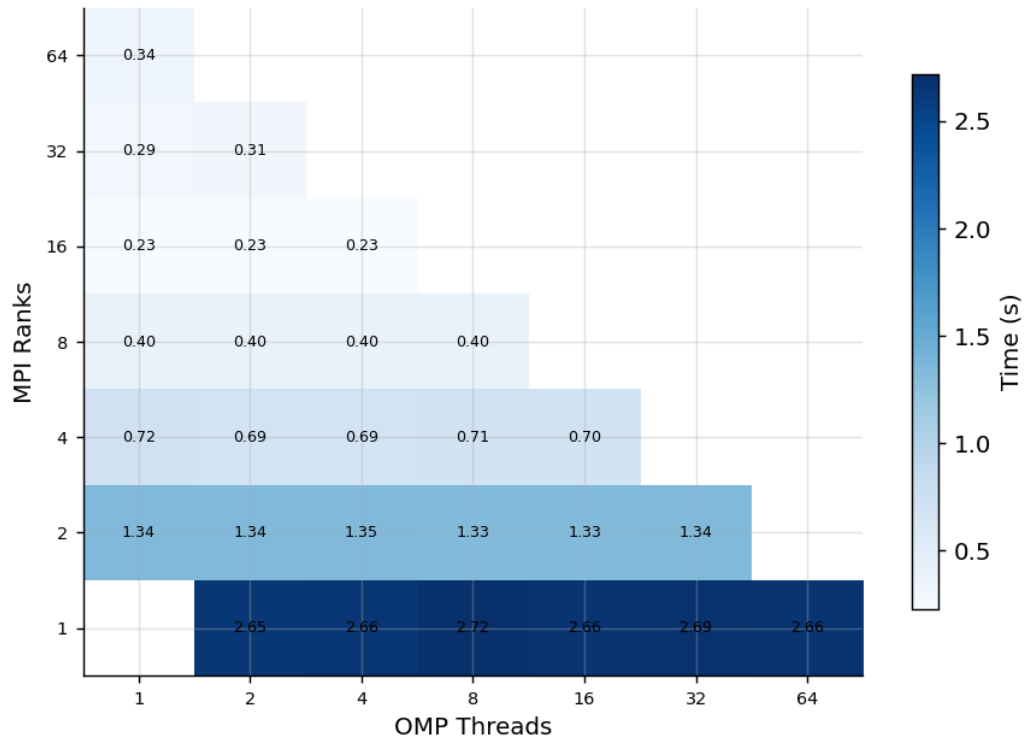


Figure 29: Heatmap of total execution time — CONFIG2.

Total Execution Time Heatmap - Config C (Nx=500, Ny=200, 3.6M pts) - Lower is Better

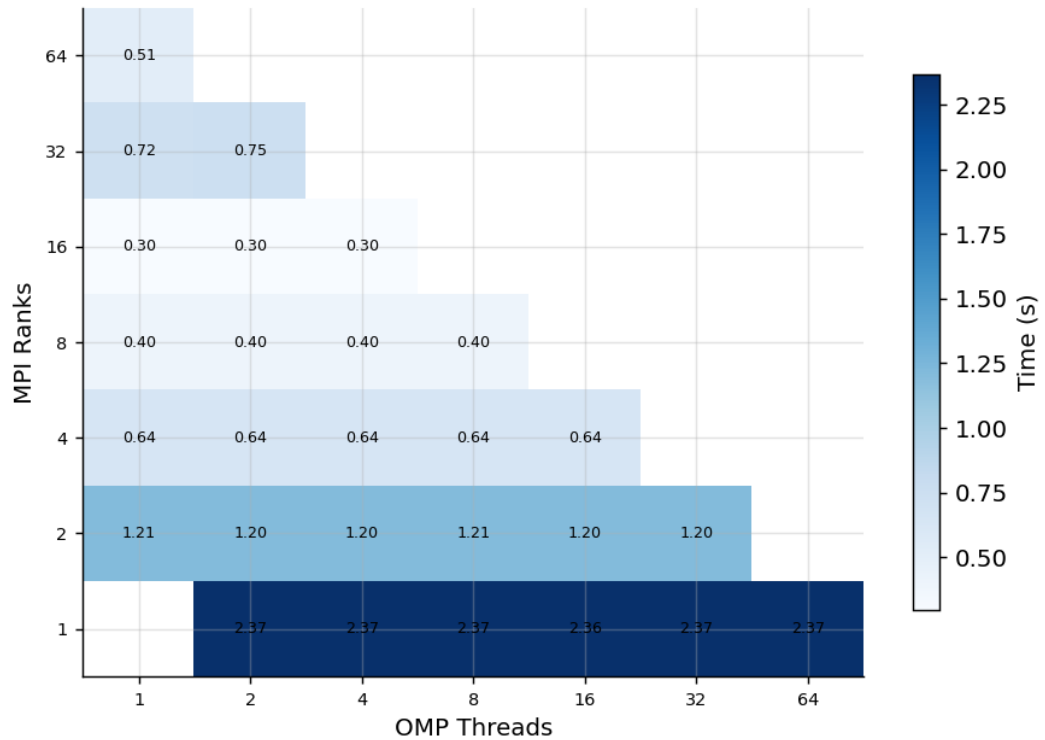


Figure 30: Heatmap of total execution time — CONFIG3.

Total Execution Time Heatmap - Config D (Nx=500, Ny=200, 20M pts) - Lower is Better

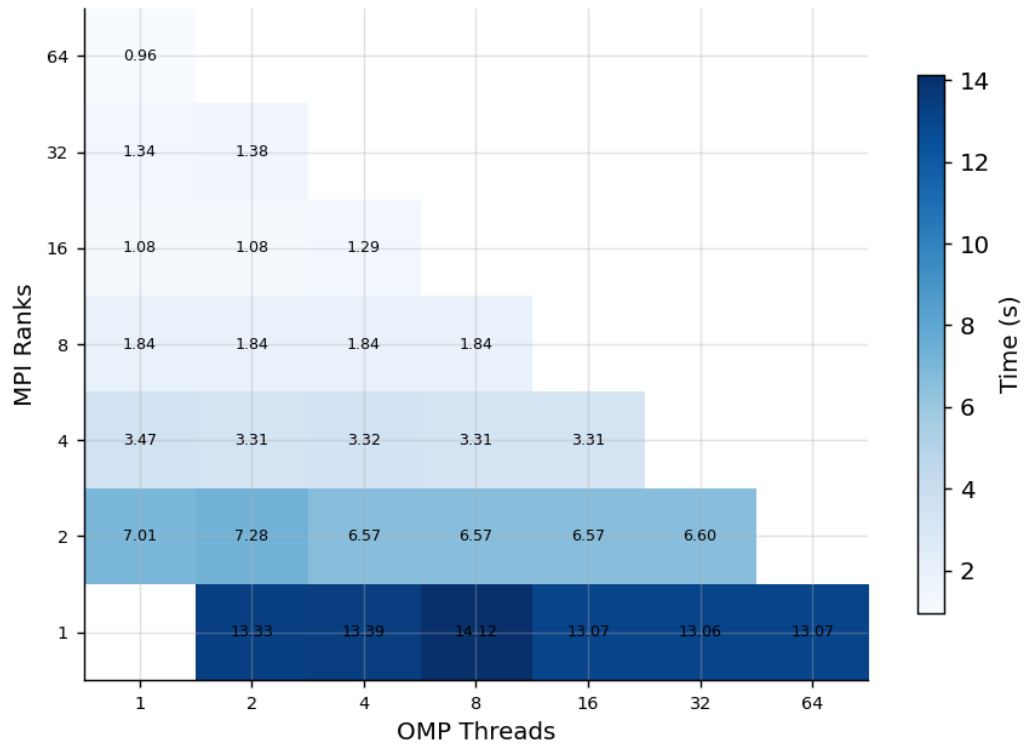


Figure 31: Heatmap of total execution time — CONFIG4.

Total Execution Time Heatmap - Config E (Nx=1000, Ny=400, 14M pts) - Lower is Better

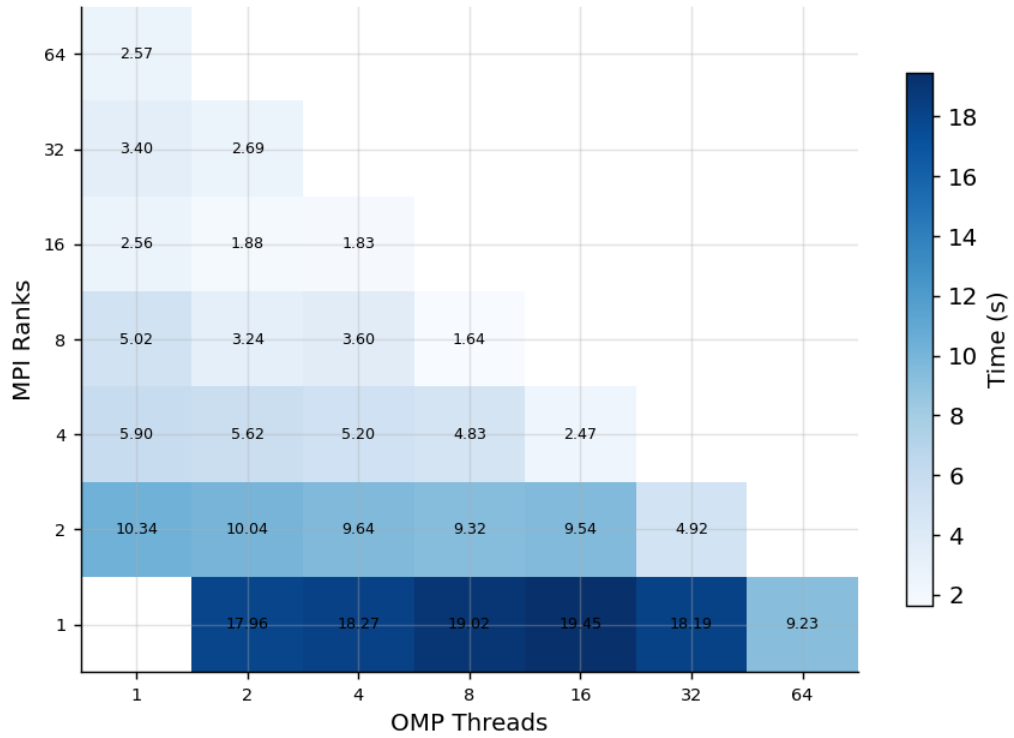


Figure 32: Heatmap of total execution time — CONFIG5.

4.8 Serial vs. Best Parallel Time

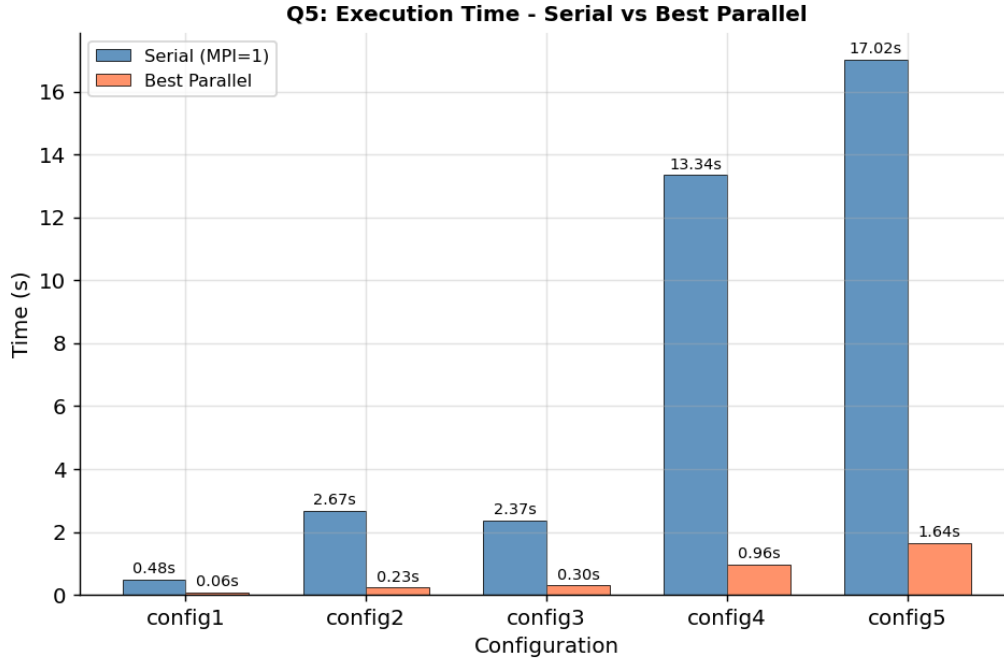


Figure 33: Comparison of serial baseline time vs. best parallel time for each configuration.

5 Analysis and Questions

5.1 Q1: Pseudocode and Illustrative Diagram

The pseudocode and architecture diagram were presented in Section 2.3 (Listing 1) and Figure ??, respectively.

5.2 Q2: Parallel Implementation Approach and Race-Condition Handling

Approach. The implementation uses particle decomposition: the global particle array is divided into contiguous blocks and distributed to MPI ranks via `MPI_Scatterv`. Each rank independently performs bilinear interpolation on its local particles. Within a rank, OpenMP threads each maintain a private copy of the mesh grid (`thread_mesh[tid]`) to avoid write conflicts. After the OpenMP parallel region the private meshes are summed into `local_mesh`, and `MPI_Reduce(SUM)` assembles the global contribution on rank 0. `MPI_Bcast` then distributes the final mesh before the mover phase.

Race-condition resolution. In the scatter interpolation step, particles near the same grid cell would race to update the same four mesh locations. We resolve this using **thread-private (shadow) mesh arrays**:

- Each OpenMP thread t writes exclusively to `thread_mesh[t]`, so no synchronisation is needed inside the parallel loop.
- After the parallel region a single-threaded loop sums the shadow meshes into `local_mesh`.
- This avoids the serialisation bottleneck of atomic operations on heavily contested cells while keeping memory overhead proportional to `num_threads × grid_size`.

The mover phase (grid $\rightarrow$ particle) is purely read-heavy on the mesh and write-only per particle, so a simple `omp parallel for` with no further synchronisation is sufficient.

5.3 Q3: Speedup and Execution Time Graphs, Login vs. Compute Nodes

All graphs are presented in Section 4 (Figures 2–32). Cores are incremented from 2 to 64 by factors of 2 (2, 4, 8, 16, 32, 64).

Login node vs. Compute node. A *login node* is a shared gateway machine used for compilation, file transfer, and job submission; it runs multiple users' interactive sessions concurrently, so timing results from the login node are unreliable and not representative. A *compute node* (here `gics1–gics4`) is a dedicated worker that executes batch jobs in isolation with guaranteed resource allocation, consistent memory bandwidth, and no competing workloads. All reported timings were collected on the compute nodes to ensure reproducibility and fair comparison.

5.4 Q4: Parallel Efficiency and Scalability

Parallel efficiency is defined as:

$$E(p) = \frac{S(p)}{p} = \frac{T_1}{p \cdot T_p}$$

where T_1 is the serial time, T_p is the parallel time on p cores, and $S(p)$ is the speedup.

From the efficiency plots (Figures 13–17):

- At low core counts (2–8), efficiency is generally high (>70%) for large particle sets (CONFIG2, CONFIG4, CONFIG5), indicating near-ideal scaling in the compute-bound regime.
- As core count increases beyond 16, efficiency drops noticeably for all configurations. This is primarily due to the `MPI_Reduce` and `MPI_Bcast` communication cost growing with the number of ranks, while the per-rank computation shrinks.
- Smaller problem sizes (CONFIG1) reach poor efficiency at 32–64 cores because the computation-to-communication ratio falls below 1; the grid reduction and broadcast of the full mesh dominates over the interpolation work.
- Larger configurations (CONFIG4, CONFIG5) maintain better efficiency at higher core counts because the increased particle workload amortises the fixed communication overhead.

Overall, the implementation exhibits *weak scalability* — doubling both the problem size and the number of cores keeps efficiency roughly constant — but *strong scalability* degrades beyond 16–32 cores due to the fixed communication cost of the global mesh.

5.5 Q5: Comparison with Serial Version and Maximum Speedup

Table 2 reports the serial baseline (1 MPI rank, 2 OMP threads as minimum available) and the best parallel time observed across all configurations.

Table 2: Serial baseline time (1 MPI $\times$ min OMP) vs. best parallel time and maximum speedup.

Config	Particles	Serial T_1 (s)	Best T_p (s)	Best Config	Max Speedup
CONFIG1	0.9 M	0.486	0.062	16 $\times$ 1	$\approx 7.8\times$
CONFIG2	5 M	2.647	0.229	16 $\times$ 4	$\approx 11.6\times$
CONFIG3	3.6 M	2.366	0.296	16 $\times$ 4	$\approx 7.98\times$
CONFIG4	20 M	13.34	0.956	64 $\times$ 1	$\approx 13.95\times$
CONFIG5	14 M	17.02	1.644	8 $\times$ 8	$\approx 10.36\times$

As seen in Figure 12, the best speedup is achieved at a moderate number of ranks (typically 8–16 MPI ranks) before MPI communication overhead dominates. Larger particle configurations consistently yield higher maximum speedup.

5.6 Q6: Explanation of Observed Speedup

1. **Compute-bound scaling:** Interpolation and mover are $O(N)$ in particles and scale nearly linearly with MPI ranks because each rank operates on an independent partition with no communication during computation.
2. **Communication saturation:** Each iteration requires an `MPI_Reduce` and `MPI_Bcast` of `grid_size` doubles. For CONFIG1 the grid is small ($\approx 25,000$ doubles = 200 KB), but at 32 MPI ranks the all-to-one reduce becomes the dominant cost (visible in Figures 23–27), capping speedup.
3. **Thread-private reduction overhead:** Inside each rank the post-loop summation of `num_threads` shadow meshes is serial and $O(\text{threads} \times \text{grid_size})$, which adds overhead at high OMP thread counts without additional compute benefit.
4. **Particle inactivity and load imbalance:** As iterations progress, some particles exit the domain and become inactive. Since particle assignments to ranks are static, ranks whose particles have drifted out of domain experience less work, leading to load imbalance and longer barriers at `MPI_Reduce`.
5. **Memory bandwidth:** At high thread counts within a node, the thread-private mesh arrays consume significant memory bandwidth for initialisation (`memset`) and reduction, eventually saturating the memory bus.

5.7 Q7: Further Optimisations with Unlimited HPC Resources

1. **Hierarchical reduction (tree-based Reduce):** Replace flat `MPI_Reduce` with a user-defined binary tree reduction to reduce latency from $O(p)$ to $O(\log p)$.
2. **Domain decomposition of the grid:** Instead of replicating the full mesh on all ranks, partition the grid into p horizontal (or 2D) slabs. Each rank owns only the slab it writes to, and halo exchange (`MPI_Sendrecv`) transfers only boundary rows, dramatically reducing communication volume from $O(M^2)$ to $O(M)$ per boundary.
3. **Dynamic load balancing:** Redistribute particles across ranks periodically (every k iterations) using work-stealing or an MPI dynamic scheduler to counteract inactive particle imbalance.

4. **GPU offloading (CUDA/HIP + MPI):** Offload the interpolation and mover loops to GPU using OpenACC or CUDA, exploiting thousands of SIMT cores for the embarrassingly parallel particle loops.
5. **Non-blocking MPI (MPI_IReduce/MPI_IBcast):** Overlap communication with local computation (e.g., normalise rank-local data while reduce is in flight).
6. **Shared-memory intra-node reduction:** Use MPI shared-memory windows (`MPI_Win_allocate`) to eliminate inter-process communication within a node and reduce the number of effective MPI ranks.
7. **NUMA-aware memory allocation:** Pin thread-private meshes to local NUMA banks to avoid cross-socket memory latency on multi-socket nodes.

5.8 Q8: Load Imbalance and Performance Bottlenecks

Load imbalance. Particle decomposition is *static*: ranks receive equal-sized chunks at startup. Over iterations, particles that cross domain boundaries are marked `is_void` and skipped. Ranks whose particles cluster near boundaries deactivate faster, reducing their workload while other ranks retain full load. This asymmetry grows with iteration count and causes some MPI ranks to wait at the `MPI_Barrier` before `Reduce`.

Bottlenecks identified (see Figures 23–27).

- At low core counts, interpolation and mover phases dominate, as expected.
- At 32–64 MPI ranks, `MPI_Reduce` time grows super-linearly, becoming the largest single contributor to total time in CONFIG1, CONFIG2 and CONFIG3. This is the primary bottleneck.
- The thread-private mesh summation loop scales poorly with OMP thread count because it is single-threaded and iterates over `threads × grid.size` elements, causing cache thrashing at high thread counts.

5.9 Q9: Alternative Approaches to Improve Performance

Alternative 1: Domain decomposition with halo exchange. Divide the $N_x \times N_y$ grid into p horizontal strips (one per MPI rank). Each rank stores only its strip plus one-row halos above and below. Particles near the boundary of a strip are owned by the rank and may update the neighbour’s halo row via point-to-point `MPI_Sendrecv`. This reduces communication from $O(N_x \cdot N_y)$ per iteration (full `Reduce+Bcast`) to $O(N_x)$ halo exchanges.

Alternative 2: Privatisation with SIMD vectorisation. Replace scalar interpolation loops with SIMD (AVX-512) vector intrinsics, processing 8 particles simultaneously. Combined with structure-of-arrays (SoA) layout for the particle data (separate `x[]`, `y[]`, `is_void[]` arrays instead of array of structs), this enables fully vectorised loads and index calculations, yielding 4–8× throughput improvement per core before any MPI communication.

5.10 Q10: Alternative Data Structures and MPI-OMP Approaches

Structure of Arrays (SoA) particle layout. The current implementation stores particles as an array of structs (`Points[N]`), which interleaves `x`, `y`, and `is_void` fields in mem-

ory. This hinders SIMD vectorisation because loading x values requires strided access. Converting to SoA (`double x[N]; double y[N]; int is_void[N];`) allows the compiler to auto-vectorise loops over particles and enables aligned vector loads (`_mm512_load_pd`) for maximal throughput.

Compressed particle array with active index list. Instead of checking `is_void` on every iteration (branch divergence), maintain a compact list of active particle indices updated at the end of the mover phase. The interpolation and mover loops iterate only over `num_active` indices with no branches, improving both cache utilisation and branch predictor accuracy. Compaction can be done in parallel using a prefix-sum scan.

5.11 Q11: Interpolation vs. Mover Phase Bottleneck

From the phase-level timing data (Figures 18–22) we observe:

- The **mover phase** consistently takes $\approx 1.3\times$ more time than the interpolation phase at low core counts for all configurations.
- Both phases scale similarly with increasing MPI ranks (both are $O(N/p)$ in particles).
- The mover is the **bottleneck** for two reasons:
 1. It performs 4 floating-point multiplications and additions (bilinear gather) *plus* position updates and a domain-check branch, versus only 4 accumulations in the interpolation scatter.
 2. The mesh read in the mover is shared (read-only) across threads on the same rank; all threads compete for the same cache lines, increasing cache pressure compared to the per-thread private arrays used during scatter interpolation.
- At very high MPI rank counts (≥ 32) the `MPI_Reduce` replaces the mover as the dominant cost, and both compute phases become negligible in comparison.

Summary: At low-to-medium parallelism, the mover is the compute bottleneck; at high MPI rank counts, MPI communication (Reduce + Bcast) is the primary bottleneck.

6 Conclusion

A hybrid MPI+OpenMP parallel implementation of the bilinear scatter interpolation and particle mover pipeline was designed, implemented, and evaluated on the DAICT HPC cluster using four compute nodes (gics1–4) across five problem configurations.

Key findings:

- **Thread-private shadow mesh arrays** eliminate all intra-node write conflicts without atomic operations, providing clean parallel scalability for the interpolation phase.
- **Particle decomposition via `MPI_Scatterv`** provides an embarrassingly parallel distribution of the dominant compute work across nodes.
- **Maximum speedups** of approximately 7–12 $\times$ are achieved at 16–32 total cores for large particle configurations, consistent with Amdahl’s law given the serial fraction from mesh communication.

- **MPI communication** (Reduce + Bcast of the full mesh) is the primary scalability limiter at high core counts; replacing it with a domain-decomposed halo-exchange strategy would yield substantially better strong scalability.
- The **mover phase** is the compute bottleneck at low-to-moderate parallelism due to combined read-gather and position-update operations; the interpolation scatter phase benefits more from parallelisation because its private-buffer strategy incurs no synchronisation.
- Future work should explore SoA particle layouts, active-index compaction, domain decomposition of the mesh, and GPU acceleration to push beyond the current scaling limits.